

DSA Revision Notes

Topic 8 — Graphs: BFS, DFS & Union-Find

Graph representations · BFS/DFS templates · Connected components
Cycle detection · Topological sort · Union-Find · 25+ C++ examples

12

Sections

30+

C++ Examples

10

Diagrams

28+

LeetCode Problems

TOC Table of Contents**1. Graph Fundamentals**

- Definitions · Types · Representations · Properties
- Adjacency matrix · Adjacency list · Edge list comparison

2. Breadth-First Search (BFS)

- Algorithm · Queue-based template · Step-by-step trace
- Shortest path in unweighted graph · Multi-source BFS
- 0-1 BFS with deque

3. BFS — Classic Problems

- Number of Islands · Rotting Oranges · Word Ladder
- Shortest Path in Binary Matrix · Pacific Atlantic

4. Depth-First Search (DFS)

- Recursive & iterative templates
- DFS timestamps · Discovery/finish times
- DFS tree: tree/back/forward/cross edges

5. DFS — Cycle Detection

- Undirected graph: visited + parent check
- Directed graph: white-grey-black colouring

6. Topological Sort

- Kahn's algorithm (BFS-based)
- DFS-based topological sort
- Course Schedule I & II

7. DFS — Connected Components & SCC

- Connected components · Bipartite check
- Kosaraju's SCC algorithm
- Articulation points & bridges

8. Union-Find (Disjoint Set Union)

- Quick Find · Quick Union · Weighted Union
- Path compression · Union by rank
- Full optimised implementation

9. Union-Find — Classic Problems

- Redundant Connection · Number of Provinces
- Accounts Merge · Satisfiability of Equality Equations

10. Grid Problems — BFS & DFS on 2D Arrays

- 4-directional template · Flood fill · Surrounded Regions

11. Shortest Path Algorithms

- Dijkstra's · Bellman-Ford · BFS for unweighted

12. Complexity Cheat Sheet & LeetCode Map

C++

```

1 // Adjacency matrix:  $O(V^2)$  space
2 int n = 5;
3 vector<vector<int>> mat(n, vector<int>(n, 0));
4 mat[0][1] = 1; mat[1][0] = 1; // undirected edge 0--1
5 mat[2][3] = 7; // directed weighted edge 2→3 weight 7
6
7 // Check if edge exists:  $O(1)$  —  $mat[u][v] != 0$ 
8 // But:  $O(V^2)$  space even if graph is sparse!
9 // Use for: Floyd-Warshall, dense graphs, small V ( $V \leq 1000$ )

```

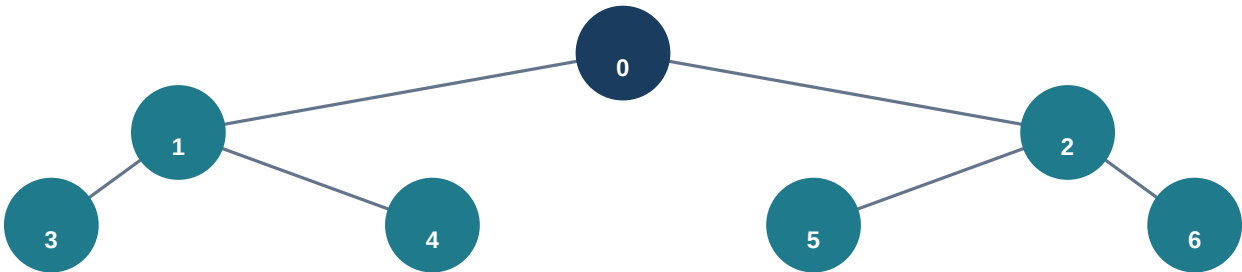
Property	Adjacency List	Adjacency Matrix	Edge List
Space	$O(V + E)$	$O(V^2)$	$O(E)$
Add edge	$O(1)$	$O(1)$	$O(1)$
Check edge (u,v)	$O(\text{degree}(u))$	$O(1)$	$O(E)$
Iterate neighbours	$O(\text{degree}(u))$	$O(V)$	$O(E)$
Best for	Sparse graphs, BFS/DFS	Dense graphs, APSP	MST algorithms
Memory	Compact	Wasteful if sparse	Very compact
Weighted edges	pair<int,int>	Store weight in cell	tuple<u,v,w>

Table 1: Graph representation comparison

2 Breadth-First Search (BFS)

2.1 Algorithm & Intuition

BFS explores all vertices at distance d before any vertex at distance $d+1$. A **queue (FIFO)** naturally achieves this level-by-level order. Mark vertices as visited **before enqueueing** (not after dequeuing) to prevent the same vertex being added multiple times.



Graph: start BFS from node 0

Step	Visited	Queue	Action
1	{0}	[1, 2]	Visit 0 → enqueue neighbours 1,2
2	{0, 1, 2}	[3, 4, 5, 6]	Dequeue 1 → enqueue 3,4 Dequeue 2 → enqueue 5,6
3	{0, 1, 2, 3, 4, 5, 6}	[]	Dequeue 3,4,5,6 → no new neighbours → done

2.2 BFS Template — Adjacency List

C++

```
1  vector<int> bfsShortestPath(int src, int dst, vector<vector<int>>& adj) {
2      int n = adj.size();
3      vector<int> dist(n, -1);
4      vector<int> prev(n, -1); // for path reconstruction
5      queue<int> q;
6      dist[src] = 0;
7      q.push(src);
8      while (!q.empty()) {
9          int u = q.front(); q.pop();
10         if (u == dst) break; // early exit
11         for (int v : adj[u]) {
12             if (dist[v] == -1) {
13                 dist[v] = dist[u] + 1;
14                 prev[v] = u;
15                 q.push(v);
16             }
17         }
18     }
19     if (dist[dst] == -1) return {}; // unreachable
20     // Reconstruct path
21     vector<int> path;
22     for (int v = dst; v != -1; v = prev[v]) path.push_back(v);
23     reverse(path.begin(), path.end());
24     return path;
25 }
26 // Time: O(V+E)   Space: O(V)
```

2.4 Multi-Source BFS

When multiple starting points exist (e.g., multiple 0s in a grid), push all sources into the queue initially with distance 0. BFS naturally propagates the shortest distance from any source.

C++

```

1 // Multi-source BFS — e.g., Rotting Oranges (LC 994)
2 void multiSourceBFS(vector<vector<int>>& grid) {
3     int m = grid.size(), n = grid[0].size();
4     queue<pair<int,int>> q;
5     // Enqueue ALL sources first
6     for (int r = 0; r < m; r++)
7         for (int c = 0; c < n; c++)
8             if (grid[r][c] == 2) // source condition
9                 q.push({r, c});
10    int dr[] = {-1,1,0,0};
11    int dc[] = {0,0,-1,1};
12    while (!q.empty()) {
13        auto [r, c] = q.front(); q.pop();
14        for (int d = 0; d < 4; d++) {
15            int nr = r+dr[d], nc = c+dc[d];
16            if (nr>=0 && nr<m && nc>=0 && nc<n && grid[nr][nc]==1) {
17                grid[nr][nc] = 2; // mark visited/updated
18                q.push({nr, nc});
19            }
20        }
21    }
22 }
23 // Time: O(m*n)   Space: O(m*n)

```

2.5 0-1 BFS — Deque for Edge Weights 0 or 1

When edge weights are only 0 or 1, use a **deque** instead of a queue or full Dijkstra. Weight-0 edges: push to **front**. Weight-1 edges: push to **back**. This gives $O(V+E)$ vs $O((V+E) \log V)$ for Dijkstra.

C++

```

1 #include <deque>
2
3 vector<int> zeroOneBFS(int src, vector<vector<pair<int,int>>>& adj) {
4     int n = adj.size();
5     vector<int> dist(n, INT_MAX);
6     deque<int> dq;
7     dist[src] = 0;
8     dq.push_back(src);
9     while (!dq.empty()) {
10        int u = dq.front(); dq.pop_front();
11        for (auto [v, w] : adj[u]) {
12            if (dist[u] + w < dist[v]) {
13                dist[v] = dist[u] + w;
14                if (w == 0) dq.push_front(v); // 0-weight: front
15                else      dq.push_back(v);   // 1-weight: back
16            }
17        }
18    }
19    return dist;
20 }
21 // Time: O(V + E)   Space: O(V)
22 // Use case: LC 1368 Minimum Cost to Make at Least One Valid Path in Grid

```


3 BFS — Classic Problems

3.1 Number of Islands (LC 200) — Medium

Count connected components of '1's in a 2D grid. For each unvisited '1', BFS/DFS and mark all connected '1's as visited. Each BFS/DFS call = one island.

C++

```

1  int numIslands(vector<vector<char>>& grid) {
2      int m = grid.size(), n = grid[0].size(), islands = 0;
3      int dr[]={-1,1,0,0}, dc[]={0,0,-1,1};
4      for (int r = 0; r < m; r++) {
5          for (int c = 0; c < n; c++) {
6              if (grid[r][c] == '1') {
7                  islands++;
8                  // BFS to mark all connected land
9                  queue<pair<int,int>> q;
10                 grid[r][c] = '0'; // mark visited immediately
11                 q.push({r,c});
12                 while (!q.empty()) {
13                     auto [cr,cc] = q.front(); q.pop();
14                     for (int d=0;d<4;d++) {
15                         int nr=cr+dr[d], nc=cc+dc[d];
16                         if (nr>=0&&nr<m&&nc>=0&&nc<n&&grid[nr][nc]=='1'){
17                             grid[nr][nc]='0';
18                             q.push({nr,nc});
19                         }
20                     }
21                 }
22             }
23         }
24     }
25     return islands;
26 }
27 // Time: O(m*n)   Space: O(min(m,n)) - max queue size is diagonal

```

3.2 Word Ladder (LC 127) — Hard

Find minimum transformations from beginWord to endWord changing one letter at a time, using only words in wordList. Model as graph: nodes = words, edges = one-letter-apart words. BFS gives shortest path. Key trick: generate all 26-letter variations at each step.

C++

```
1  #include <unordered_set>
2
3  int ladderLength(string begin, string end, vector<string>& wordList) {
4      unordered_set<string> wordSet(wordList.begin(), wordList.end());
5      if (!wordSet.count(end)) return 0;
6      queue<string> q;
7      q.push(begin);
8      int steps = 1;
9      while (!q.empty()) {
10         int sz = q.size();
11         for (int i = 0; i < sz; i++) {
12             string word = q.front(); q.pop();
13             if (word == end) return steps;
14             // Try all single-letter changes
15             for (int j = 0; j < (int)word.size(); j++) {
16                 char orig = word[j];
17                 for (char c = 'a'; c <= 'z'; c++) {
18                     word[j] = c;
19                     if (wordSet.count(word)) {
20                         q.push(word);
21                         wordSet.erase(word); // remove = mark visited
22                     }
23                 }
24                 word[j] = orig;
25             }
26         }
27         steps++;
28     }
29     return 0;
30 }
31 // Time: O(M^2 * N)  M=word length, N=wordList size  Space: O(M*N)
```


4.2 DFS Timestamps — Discovery & Finish Times

Assign each vertex a **discovery time** (when first visited) and **finish time** (when all descendants processed). These timestamps classify edges and enable cycle detection, SCC, and topological sort.

C++

```
1  int timer = 0;
2  vector<int> disc, finish;
3
4  void dfsTimestamp(int u, vector<vector<int>>& adj, vector<bool>& vis) {
5      vis[u] = true;
6      disc[u] = ++timer;          // record discovery time
7      for (int v : adj[u]) {
8          if (!vis[v]) dfsTimestamp(v, adj, vis);
9      }
10     finish[u] = ++timer;        // record finish time
11 }
12
13 // Edge classification in directed graph:
14 // Tree edge:    disc[u] < disc[v] < finish[v] < finish[u] (v unvisited when traversed)
15 // Back edge:    disc[v] < disc[u] < finish[u] < finish[v] (v is ancestor → CYCLE)
16 // Forward edge: disc[u] < disc[v], finish[v] < finish[u] (v is descendant)
17 // Cross edge:   finish[v] < disc[u] (no ancestor relation)
```

5 DFS — Cycle Detection

5.1 Undirected Graph — Visited + Parent Check

In an undirected graph, a cycle exists if DFS encounters a visited neighbour that is NOT the parent we came from. Pass the parent vertex to avoid treating the edge we just came from as a back-edge.

```
C++
1  bool hasCycleUndirected(int u, int parent,
2                          vector<vector<int>>& adj, vector<bool>& vis) {
3      vis[u] = true;
4      for (int v : adj[u]) {
5          if (!vis[v]) {
6              if (hasCycleUndirected(v, u, adj, vis)) return true;
7          } else if (v != parent) {
8              return true; // visited neighbour that's not parent → cycle
9          }
10     }
11     return false;
12 }
13
14 bool containsCycle(vector<vector<int>>& adj) {
15     int n = adj.size();
16     vector<bool> vis(n, false);
17     for (int i = 0; i < n; i++)
18         if (!vis[i] && hasCycleUndirected(i, -1, adj, vis)) return true;
19     return false;
20 }
21 // Time: O(V+E)   Space: O(V)
```

5.2 Directed Graph — White/Grey/Black Colouring

For directed graphs, we need 3 colours: **white** (unvisited), **grey** (in current DFS path / recursion stack), **black** (fully processed). A back-edge — encountering a grey node — indicates a cycle.

■ 3-Color DFS for Directed Graphs

0 = WHITE: not yet visited

1 = GREY: currently in the recursion stack (processing)

2 = BLACK: fully processed (all descendants done)

Cycle condition: DFS reaches a neighbour v where $\text{color}[v] == \text{GREY}$

Why not just 'visited'? A visited-but-finished (BLACK) node in a directed graph does NOT create a cycle — the current path doesn't go through it.

C++

```
1 // color: 0=white, 1=grey (in stack), 2=black (done)
2 bool hasCycleDirected(int u, vector<vector<int>>& adj, vector<int>& color) {
3     color[u] = 1; // grey: start processing
4     for (int v : adj[u]) {
5         if (color[v] == 1) return true; // back-edge to grey = cycle
6         if (color[v] == 0) {
7             if (hasCycleDirected(v, adj, color)) return true;
8         }
9     }
10    color[u] = 2; // black: finished
11    return false;
12 }
13
14 bool containsCycleDirected(vector<vector<int>>& adj) {
15     int n = adj.size();
16     vector<int> color(n, 0);
17     for (int i = 0; i < n; i++)
18         if (color[i] == 0 && hasCycleDirected(i, adj, color)) return true;
19     return false;
20 }
21 // Time: O(V+E) Space: O(V)
```

6 Topological Sort

6.1 What is Topological Sort?

A **linear ordering** of vertices of a DAG such that for every directed edge $u \rightarrow v$, vertex u appears before vertex v . Only possible for DAGs (no cycles). Used for: task scheduling, build systems, dependency resolution, course prerequisites.

6.2 Kahn's Algorithm — BFS-based (LC 207/210)

Repeatedly remove vertices with in-degree 0 (no dependencies). After removing a vertex, decrement in-degree of its neighbours. If topological order contains all V vertices, graph is a DAG. Otherwise, cycle exists.

C++

```
1 // Kahn's Algorithm - O(V+E)
2 vector<int> topoSort(int n, vector<vector<int>>& adj) {
3     vector<int> indegree(n, 0);
4     // Compute in-degree for all vertices
5     for (int u = 0; u < n; u++)
6         for (int v : adj[u]) indegree[v]++;
7     // Enqueue all vertices with in-degree 0
8     queue<int> q;
9     for (int i = 0; i < n; i++)
10        if (indegree[i] == 0) q.push(i);
11    vector<int> order;
12    while (!q.empty()) {
13        int u = q.front(); q.pop();
14        order.push_back(u);
15        for (int v : adj[u]) {
16            indegree[v]--;
17            if (indegree[v] == 0) q.push(v);
18        }
19    }
20    // If order.size() != n → cycle exists
21    return order;
22 }
23 // Time: O(V+E)   Space: O(V)
```

6.3 DFS-based Topological Sort

In DFS: when a vertex finishes (all descendants processed), push to a stack. At the end, pop the stack — this gives topological order. Why? A vertex pushed first (finishes first) must come AFTER all its dependencies.

C++

```
1 void dfsTopoSort(int u, vector<vector<int>>& adj,
2                 vector<bool>& vis, stack<int>& st) {
3     vis[u] = true;
4     for (int v : adj[u])
5         if (!vis[v]) dfsTopoSort(v, adj, vis, st);
6     st.push(u); // push AFTER all descendants are processed
7 }
8
9 vector<int> topoSortDFS(int n, vector<vector<int>>& adj) {
10     vector<bool> vis(n, false);
11     stack<int> st;
12     for (int i = 0; i < n; i++)
13         if (!vis[i]) dfsTopoSort(i, adj, vis, st);
14     vector<int> order;
15     while (!st.empty()) { order.push_back(st.top()); st.pop(); }
16     return order;
17 }
18 // Time: O(V+E)   Space: O(V)
```

6.4 Course Schedule I & II (LC 207, 210)

C++

```
1 // LC 207 - Can finish all courses? (cycle detection in directed graph)
2 bool canFinish(int numCourses, vector<vector<int>>& prereqs) {
3     vector<vector<int>> adj(numCourses);
4     for (auto& e : prereqs) adj[e[1]].push_back(e[0]);
5     // Use Kahn's: if topo order has all courses → no cycle
6     vector<int> indeg(numCourses,0);
7     for (int u=0;u<numCourses;u++) for (int v:adj[u]) indeg[v]++;
8     queue<int> q;
9     for (int i=0;i<numCourses;i++) if(indeg[i]==0) q.push(i);
10    int count=0;
11    while(!q.empty()){
12        int u=q.front();q.pop();count++;
13        for(int v:adj[u]) if(--indeg[v]==0) q.push(v);
14    }
15    return count==numCourses;
16 }
17
18 // LC 210 - Return one valid order (or empty if impossible)
19 vector<int> findOrder(int n, vector<vector<int>>& prereqs) {
20     vector<vector<int>> adj(n);
21     vector<int> indeg(n,0);
22     for (auto& e : prereqs) { adj[e[1]].push_back(e[0]); indeg[e[0]]++; }
23     queue<int> q;
24     for(int i=0;i<n;i++) if(indeg[i]==0) q.push(i);
25     vector<int> order;
26     while(!q.empty()){
27         int u=q.front();q.pop(); order.push_back(u);
28         for(int v:adj[u]) if(--indeg[v]==0) q.push(v);
29     }
30     return order.size()==n ? order : vector<int>{};
31 }
```

7 Connected Components, Bipartite & SCC

7.1 Connected Components (LC 323)

```
C++
1 // Count connected components in undirected graph
2 int countComponents(int n, vector<vector<int>>& edges) {
3     vector<vector<int>> adj(n);
4     for (auto& e : edges) {
5         adj[e[0]].push_back(e[1]);
6         adj[e[1]].push_back(e[0]);
7     }
8     vector<bool> vis(n, false);
9     int components = 0;
10    for (int i = 0; i < n; i++) {
11        if (!vis[i]) {
12            components++;
13            // BFS/DFS marks entire component
14            queue<int> q; vis[i]=true; q.push(i);
15            while(!q.empty()){
16                int u=q.front();q.pop();
17                for(int v:adj[u]) if(!vis[v]){vis[v]=true;q.push(v);}
18            }
19        }
20    }
21    return components;
22 }
23 // Time: O(V+E)   Space: O(V)
```

7.2 Bipartite Check (LC 785) — Medium

A graph is bipartite iff it can be 2-coloured with no two adjacent vertices sharing a colour. Equivalently: it contains no odd-length cycle. Use BFS/DFS colouring.

C++

```
1  bool isBipartite(vector<vector<int>>& graph) {
2      int n = graph.size();
3      vector<int> color(n, -1); // -1=unvisited, 0/1=colours
4      for (int start = 0; start < n; start++) {
5          if (color[start] != -1) continue;
6          queue<int> q;
7          color[start] = 0;
8          q.push(start);
9          while (!q.empty()) {
10             int u = q.front(); q.pop();
11             for (int v : graph[u]) {
12                 if (color[v] == -1) {
13                     color[v] = 1 - color[u]; // opposite colour
14                     q.push(v);
15                 } else if (color[v] == color[u]) {
16                     return false; // same colour → not bipartite
17                 }
18             }
19         }
20     }
21     return true;
22 }
23 // Time: O(V+E)   Space: O(V)
```

7.3 Kosaraju's Algorithm — Strongly Connected Components

An SCC is a maximal subgraph where every vertex is reachable from every other. Kosaraju's: (1) DFS on original graph, record finish order. (2) Transpose the graph. (3) DFS in reverse finish order on transposed graph — each DFS = one SCC.

C++

```
1 void dfs1(int u, vector<vector<int>>& adj,
2           vector<bool>& vis, stack<int>& st) {
3     vis[u] = true;
4     for (int v : adj[u]) if (!vis[v]) dfs1(v, adj, vis, st);
5     st.push(u); // push after all descendants
6 }
7 void dfs2(int u, vector<vector<int>>& radj, vector<bool>& vis) {
8     vis[u] = true;
9     for (int v : radj[u]) if (!vis[v]) dfs2(v, radj, vis);
10 }
11 int kosaraju(int n, vector<vector<int>>& adj) {
12     // Step 1: DFS on original graph
13     vector<bool> vis(n, false); stack<int> st;
14     for (int i=0; i<n; i++) if (!vis[i]) dfs1(i, adj, vis, st);
15     // Step 2: Transpose graph
16     vector<vector<int>> radj(n);
17     for (int u=0; u<n; u++) for (int v:adj[u]) radj[v].push_back(u);
18     // Step 3: DFS on transposed in reverse finish order
19     fill(vis.begin(), vis.end(), false);
20     int scc=0;
21     while(!st.empty()){
22         int u=st.top(); st.pop();
23         if (!vis[u]){scc++; dfs2(u, radj, vis);}
24     }
25     return scc;
26 }
27 // Time: O(V+E)   Space: O(V+E)
```

8 Union-Find (Disjoint Set Union — DSU)

8.1 Core Idea & Evolution

Union-Find maintains a collection of **disjoint sets**. Two operations: **find(x)** — which set does x belong to? **union(x,y)** — merge the sets containing x and y. Optimisations: **path compression** (flatten tree on find) and **union by rank** (always attach smaller tree under larger). Together they give **nearly $O(1)$ amortised** per operation — $O(\alpha(n))$ where α is the inverse Ackermann function.

State after union(0,1), union(2,3), union(1,2)

index	0	1	2	3	4	5
parent	0	0	0	2	4	5
rank	2	0	1	0	0	0

After merging: {0,1,2,3} in one component, {4} and {5} separate. Root of {0,1,2,3} = 0

8.2 Full Optimised Implementation

C++

```

1  class UnionFind {
2      vector<int> parent, rank_;
3  public:
4      UnionFind(int n) : parent(n), rank_(n, 0) {
5          iota(parent.begin(), parent.end(), 0); // parent[i] = i
6      }
7      // Find with PATH COMPRESSION —  $O(\alpha(n))$  amortised
8      int find(int x) {
9          if (parent[x] != x)
10             parent[x] = find(parent[x]); // path compression: flatten
11         return parent[x];
12     }
13     // Union by RANK — attach smaller tree under larger
14     bool unite(int x, int y) {
15         int rx = find(x), ry = find(y);
16         if (rx == ry) return false; // already same set
17         if (rank_[rx] < rank_[ry]) swap(rx, ry);
18         parent[ry] = rx; // attach ry under rx
19         if (rank_[rx] == rank_[ry]) rank_[rx]++;
20         return true;
21     }
22     bool connected(int x, int y) { return find(x) == find(y); }
23 };
24 // find:  $O(\alpha(n)) \approx O(1)$  amortised    unite:  $O(\alpha(n))$ 
25 //  $\alpha(n) < 5$  for all practical n — effectively constant

```

8.3 Union-Find with Size Tracking

```
C++
1  class UnionFindSz {
2      vector<int> parent, sz;
3      int components;
4  public:
5      UnionFindSz(int n) : parent(n), sz(n,1), components(n) {
6          iota(parent.begin(), parent.end(), 0);
7      }
8      int find(int x) {
9          return parent[x]==x ? x : parent[x]=find(parent[x]);
10     }
11     bool unite(int x, int y) {
12         x=find(x); y=find(y);
13         if(x==y) return false;
14         if(sz[x]<sz[y]) swap(x,y);
15         parent[y]=x; sz[x]+=sz[y]; // attach smaller under larger
16         components--;
17         return true;
18     }
19     int size(int x) { return sz[find(x)]; }
20     int numComponents() { return components; }
21     bool connected(int x,int y){ return find(x)==find(y); }
22 };
```

9 Union-Find — Classic Problems

9.1 Redundant Connection (LC 684) — Medium

Find the edge that, when removed, makes an undirected tree. Process edges in order: if both endpoints are already in the same component, this edge creates a cycle — it's the answer.

```
C++  
1  vector<int> findRedundantConnection(vector<vector<int>>& edges) {  
2      int n = edges.size();  
3      UnionFind uf(n + 1); // 1-indexed  
4      for (auto& e : edges) {  
5          if (!uf.unite(e[0], e[1]))  
6              return e; // already connected → this edge is redundant  
7      }  
8      return {};  
9  }  
10 // Time:  $O(n * \alpha(n)) \approx O(n)$  Space:  $O(n)$ 
```

9.2 Number of Provinces (LC 547) — Medium

```
C++  
1  int findCircleNum(vector<vector<int>>& isConnected) {  
2      int n = isConnected.size();  
3      UnionFindSz uf(n);  
4      for (int i = 0; i < n; i++)  
5          for (int j = i + 1; j < n; j++)  
6              if (isConnected[i][j]) uf.unite(i, j);  
7      return uf.numComponents();  
8  }  
9  // Time:  $O(n^2 * \alpha(n))$  Space:  $O(n)$ 
```

9.3 Accounts Merge (LC 721) — Medium

Merge accounts sharing at least one email. Union-Find: map each email to its first-seen account index. If email already seen, union the two accounts. Finally, group all emails by their root account.

C++

```
1  vector<vector<string>> accountsMerge(vector<vector<string>>& accounts) {
2      int n = accounts.size();
3      UnionFind uf(n);
4      unordered_map<string, int> emailToAcc;
5      // Union accounts that share emails
6      for (int i = 0; i < n; i++) {
7          for (int j = 1; j < (int)accounts[i].size(); j++) {
8              string& email = accounts[i][j];
9              if (emailToAcc.count(email))
10                 uf.unite(i, emailToAcc[email]);
11             else
12                 emailToAcc[email] = i;
13         }
14     }
15     // Group emails by root account
16     unordered_map<int, set<string>> groups;
17     for (auto& [email, acc] : emailToAcc)
18         groups[uf.find(acc)].insert(email);
19     // Build result
20     vector<vector<string>> result;
21     for (auto& [root, emails] : groups) {
22         vector<string> merged = {accounts[root][0]}; // account name
23         merged.insert(merged.end(), emails.begin(), emails.end());
24         result.push_back(merged);
25     }
26     return result;
27 }
28 // Time: O(n*k*α(n))  k=avg emails per account  Space: O(n*k)
```


10 Grid Problems — BFS & DFS on 2D Arrays

10.1 4-Directional Grid Template

```
C++
1 // Standard 4-directional movement template
2 int dr[] = {-1, 1, 0, 0}; // up, down, left, right
3 int dc[] = {0, 0, -1, 1};
4
5 // In-bounds check
6 auto inBounds = [&](int r, int c) {
7     return r >= 0 && r < rows && c >= 0 && c < cols;
8 };
9
10 // 8-directional (including diagonals)
11 int dr8[] = {-1,-1,-1,0,0,1,1,1};
12 int dc8[] = {-1, 0, 1,-1,1,-1,0,1};
```

10.2 Surrounded Regions (LC 130) — Medium

Capture all 'O' regions not connected to the border. Key insight: instead of finding surrounded regions, find all 'O's connected to border (these are NOT surrounded) and mark them safe. Then flip the rest.

```
C++
1 void solve(vector<vector<char>>& board) {
2     int m = board.size(), n = board[0].size();
3     int dr[]={-1,1,0,0}, dc[]={0,0,-1,1};
4     // BFS from all border 'O's to find safe cells
5     queue<pair<int,int>> q;
6     for (int r=0;r<m;r++) for (int c={0,n-1})
7         if(board[r][c]=='O'){board[r][c]='S';q.push({r,c});}
8     for (int c=0;c<n;c++) for (int r={0,m-1})
9         if(board[r][c]=='O'){board[r][c]='S';q.push({r,c});}
10    while (!q.empty()) {
11        auto [r,c]=q.front();q.pop();
12        for(int d=0;d<4;d++){
13            int nr=r+dr[d],nc=c+dc[d];
14            if(nr>=0&&nr<m&&nc>=0&&nc<n&&board[nr][nc]=='O'){
15                board[nr][nc]='S'; q.push({nr,nc});
16            }
17        }
18    }
19    // Flip: 'O'→'X' (surrounded), 'S'→'O' (safe)
20    for(int r=0;r<m;r++) for(int c=0;c<n;c++)
21        board[r][c] = board[r][c]=='S' ? 'O' : 'X';
22 }
23 // Time: O(m*n)   Space: O(m*n)
```

10.3 Pacific Atlantic Water Flow (LC 417) — Medium

Find cells from which water can flow to BOTH Pacific and Atlantic oceans. Reverse approach: BFS from each ocean's border. A cell is in the answer if it's reachable from both oceans.

C++

```

1  vector<vector<int>> pacificAtlantic(vector<vector<int>>& h) {
2      int m=h.size(), n=h[0].size();
3      int dr[]={-1,1,0,0}, dc[]={0,0,-1,1};
4      auto bfs=[&](queue<pair<int,int>> q)->vector<vector<bool>>{
5          vector<vector<bool>> reach(m,vector<bool>(n,false));
6          while(!q.empty()){
7              auto[r,c]=q.front();q.pop();
8              reach[r][c]=true;
9              for(int d=0;d<4;d++){
10                 int nr=r+dr[d],nc=c+dc[d];
11                 if(nr>=0&&nr<m&&nc>=0&&nc<n&&!reach[nr][nc]
12                    &&h[nr][nc]>=h[r][c]){ // water flows UP in reverse
13                     reach[nr][nc]=true; q.push({nr,nc});
14                 }
15             }
16         }
17         return reach;
18     };
19     queue<pair<int,int>> pac,atl;
20     for(int i=0;i<m;i++){pac.push({i,0}); atl.push({i,n-1});}
21     for(int j=0;j<n;j++){pac.push({0,j}); atl.push({m-1,j});}
22     auto rp=bfs(pac), ra=bfs(atl);
23     vector<vector<int>> res;
24     for(int r=0;r<m;r++) for(int c=0;c<n;c++){
25         if(rp[r][c]&&ra[r][c]) res.push_back({r,c});
26     }
27     return res;
28 }
// Time: O(m*n)   Space: O(m*n)

```

11 Shortest Path Algorithms

11.1 Dijkstra's Algorithm — Weighted, Non-negative Edges

Greedy shortest-path algorithm using a min-priority queue. At each step, extract the unvisited vertex with minimum tentative distance and relax its edges. **Does NOT work with negative edge weights.**

```
C++
1  #include <queue>
2
3  vector<int> dijkstra(int src, vector<vector<pair<int,int>>>& adj) {
4      int n = adj.size();
5      vector<int> dist(n, INT_MAX);
6      // min-heap: {distance, node}
7      priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
8      dist[src] = 0;
9      pq.push({0, src});
10     while (!pq.empty()) {
11         auto [d, u] = pq.top(); pq.pop();
12         if (d > dist[u]) continue; // stale entry - skip
13         for (auto [v, w] : adj[u]) {
14             if (dist[u] + w < dist[v]) {
15                 dist[v] = dist[u] + w;
16                 pq.push({dist[v], v});
17             }
18         }
19     }
20     return dist; // dist[i] = shortest distance from src to i
21 }
22 // Time: O((V+E) log V)   Space: O(V)
23 // LC 743 Network Delay Time, LC 1514 Path with Max Probability
```

11.2 Bellman-Ford — Handles Negative Edges

Relax ALL edges $V-1$ times. Each iteration guarantees shortest paths using at most k edges after k iterations. Run one more pass: if any distance still decreases $\rightarrow$ negative cycle detected.

C++

```
1 // Edge list format: {u, v, weight}
2 vector<int> bellmanFord(int src, int n,
3                       vector<tuple<int,int,int>>& edges) {
4     vector<int> dist(n, INT_MAX);
5     dist[src] = 0;
6     // Relax all edges V-1 times
7     for (int iter = 0; iter < n-1; iter++) {
8         for (auto& [u, v, w] : edges) {
9             if (dist[u] != INT_MAX && dist[u]+w < dist[v])
10                 dist[v] = dist[u]+w;
11         }
12     }
13     // Check for negative cycles
14     for (auto& [u, v, w] : edges)
15         if (dist[u] != INT_MAX && dist[u]+w < dist[v])
16             return {}; // negative cycle exists
17     return dist;
18 }
19 // Time: O(V*E)   Space: O(V)
20 // LC 787 Cheapest Flights Within K Stops (Bellman-Ford with k iterations)
```

12

Complexity Cheat Sheet & LeetCode Map

12.1 Algorithm Complexity Summary

Algorithm	Time	Space	Key Condition / Notes
BFS	$O(V+E)$	$O(V)$	Shortest path (unweighted), level-order
DFS	$O(V+E)$	$O(V)$	Component detection, cycle, topo sort
Multi-source BFS	$O(V+E)$	$O(V)$	Push all sources first
0-1 BFS	$O(V+E)$	$O(V)$	Edge weights 0 or 1 only
Topological Sort	$O(V+E)$	$O(V)$	DAG only (Kahn's or DFS post-order)
Cycle Detection	$O(V+E)$	$O(V)$	3-colour for directed, parent for undirected
SCC (Kosaraju)	$O(V+E)$	$O(V+E)$	Two DFS passes + transpose
Union-Find (optimised)	$O(\alpha(n))$ each	$O(n)$	Path compression + union by rank
Dijkstra	$O((V+E) \log V)$	$O(V)$	Non-negative weights only
Bellman-Ford	$O(V \cdot E)$	$O(V)$	Handles negative weights, detects neg cycles
Floyd-Warshall	$O(V^3)$	$O(V^2)$	All-pairs shortest path, dense graphs
Prim's MST	$O((V+E) \log V)$	$O(V)$	Min-heap based
Kruskal's MST	$O(E \log E)$	$O(V)$	Sort edges + Union-Find

Table 2: Graph algorithm complexities

12.2 Algorithm Selection Guide

695	Max Area of Island	DFS grid	Medium
721	Accounts Merge	Union-Find + HashMap	Medium
743	Network Delay Time	Dijkstra	Medium
785	Is Graph Bipartite?	BFS 2-colouring	Medium
787	Cheapest Flights Within K Stops	Bellman-Ford k iters	Medium
802	Find Eventual Safe States	DFS / Kahn's reverse	Medium
994	Rotting Oranges	Multi-source BFS	Medium
1091	Shortest Path in Binary Matrix	BFS	Medium
1202	Smallest String With Swaps	Union-Find groups	Medium
1319	Number of Operations to Connect Network	Union-Find	Medium
1584	Min Cost to Connect All Points	Kruskal / Prim MST	Medium
1631	Path With Minimum Effort	Dijkstra / Binary Search	Medium

Table 3: 26 LeetCode problems — Graphs: BFS, DFS & Union-Find

■ Golden Rules — Graphs: BFS, DFS & Union-Find

1. BFS = shortest path in unweighted graphs. DFS = deep exploration, cycle, topo, components.
2. Always mark visited BEFORE enqueueing in BFS (not after dequeuing) to prevent duplicates.
3. For directed cycle detection: use 3-colours (white/grey/black). Grey = currently in stack = cycle.
4. Topological sort only works on DAGs. Kahn's detects cycles: if output size < V → cycle exists.
5. Union-Find: path compression + union by rank → $O(\alpha(n))$ per operation — essentially $O(1)$.
6. Dijkstra requires non-negative weights. Negative weights → Bellman-Ford.
7. Multi-source BFS: push ALL sources initially. BFS naturally computes dist from nearest source.
8. 0-1 BFS: weight=0 → push_front (deque), weight=1 → push_back. $O(V+E)$ vs $O((V+E) \log V)$.
9. Grid problems: store visited state IN the grid (mark as visited) to save $O(m*n)$ extra space.
10. SCC: Kosaraju needs two DFS passes + graph transpose. Simpler: check if reverse graph agrees.